Donghyung Lee

DevOps Engineer | Career Description

Core Competencies

Cloud Infrastructure Design & Operations

Multi/hybrid cloud architecture design and operations experience on AWS and GCP

- AWS EKS, GCP GKE production operations & cost optimization (monthly $60,000 → $48,000, 50% reduction achievements)
- IaC-based infrastructure automation and version control using Terraform (100% GCP resource codification)
- On-premises to cloud hybrid architecture design and migration experience

CI/CD Pipeline & GitOps

Maximizing development productivity through GitOps-based deployment automation

- CI/CD pipeline design and enhancement using GitHub Actions, GitLab CI, ArgoCD
- 50-67% deployment time reduction, 95% rollback time decrease (30min → 1-2min)
- Jenkins-based Unity Android/iOS build automation and Slack-integrated deployment notifications

Monitoring & Observability System Implementation

Establishing proactive incident detection and rapid response systems

- Integrated metric/log monitoring with PLG Stack (Prometheus, Loki, Grafana)
- Centralized logging system with ELK Stack (Elasticsearch, Logstash, Kibana, Filebeat)
- 60-70% incident response time reduction, 99.9% service availability achieved

Automation & Efficiency

Operations automation and developer productivity tools using Python and Bash

- 90%+ manual work reduction through data collection, deployment, and document management automation
- Improved dev team productivity by building internal LLM service (Ollama + Open WebUI)
- Internal tool development: GitLab Webhook doc sync, Slack Bot APK deployment notifications

Professional Experience

Phantom (Concrit Studio)

2024.10 ~ Present

DevOps Engineer

As a dedicated DevOps infrastructure engineer at a game development company, I design, build, and operate AWS-based cloud and on-premises servers. Development and test servers are on-premises, while QA and Production servers are on AWS, operating a hybrid architecture.

Currently providing secondary technical support and operations for the live game 'Souls' in collaboration with the publisher, and building on-premises/AWS infrastructure to improve development productivity for new game projects.

Project 1: AWS-On-premises Hybrid Cloud Architecture

Contribution 100% (Solo design & implementation)   6 months (2024.10 ~)

Problem

While operating the entire infrastructure on AWS, high cloud costs of $60,000/month necessitated cost optimization.

Approach

Workload-specific cost analysis revealed that dev/test environments had minimal traffic variation, making cloud elasticity unnecessary, while only QA/Production required scalability and stability. Designed a hybrid architecture transitioning dev/test to on-premises while keeping QA/Production on AWS. Both environments operate independently without VPN connections, with clearly separated roles to prevent failure propagation.

Troubleshooting

- When configuring ALB Ingress on EKS, needed to route to different backends per game client version. Resolved by combining custom header-based routing rules with Target Groups in ALB conditional routing
- Intermittent timeouts during ClusterIP service communication in Kubernetes IPVS mode. Resolved by analyzing Cilium eBPF packet flow and adjusting IPVS connection tracking table settings
- Pod scheduling delays during EFS mounting. Resolved by optimizing EFS CSI Driver mount options and adjusting StorageClass settings

Results

- 20% monthly infra cost reduction ($60,000 → $48,000), ~$144,000 annual savings
- Optimized AWS resource usage by completing on-premises migration of dev/test environments
- Ensured operational stability and prevented failure propagation through environment role separation

Tech Stack

AWS (EKS, EC2, ALB, Route53, ACM, EFS, Aurora MySQL, ElastiCache, CloudFront), Kubernetes, Terraform, Helm

---

Project 2: On-premises CI/CD Pipeline Construction & Enhancement

Contribution 100% (Full infra & pipeline ownership)   6 months (2024.12 ~)

Problem

Developers manually built and deployed to servers, taking an average of 30 minutes per deployment with frequent failures due to human errors.

Approach

Built a GitOps-based automated deployment environment combining GitLab CI and ArgoCD. Developers simply Git Push to trigger automatic build→test→deploy, with ArgoCD monitoring Kubernetes cluster state for declarative deployments. Eliminated Docker-in-Docker dependency using Kaniko for Docker image builds, and configured multi-environment (alpha/review/dev/staging) deployments in a single pipeline.

Troubleshooting

- TLS handshake failure with Harbor registry due to OpenSSL version mismatch in Alpine-based containers in GitLab CI pipeline. Traced the cause and resolved by replacing base image with OpenSSL 3.x compatible version
- Package installation failure on CI Runner due to GPG key expiration after GitLab upgrade. Automated GPG key renewal process to prevent recurrence
- Build time spikes due to cache invalidation during Kaniko builds. Stabilized build times by combining -cache=true --cache-ttl=24h --snapshot-mode=redo options to improve cache hit rate

Results

- 67% deployment time reduction (30min → 10min)
- 90% manual work automation (build, deploy, config apply)
- 3x+ weekly deployments (1-2 → 5-7), improved dev productivity through shorter release cycles

Tech Stack

GitLab CI/CD, ArgoCD, Jenkins, Kaniko, Kubernetes, Docker, Harbor

---

Project 3: ELK-based Integrated Monitoring & Logging System

Contribution 100% (System design & implementation)   4 months (2025.06 ~)

Problem

Server and container logs were distributed across the on-premises environment, making root cause analysis time-consuming during incidents, with no unified monitoring system in place.

Approach

Built a centralized logging system using Elasticsearch, Logstash, Kibana, and Filebeat. Collected logs from all servers and containers via Filebeat, processed JSON parsing and field mapping in Logstash, stored in Elasticsearch, enabling real-time monitoring and search through Kibana dashboards.

Troubleshooting

- Logstash parsing failures due to null bytes (\x00) in game server logs. Resolved by adding a preprocessing step using gsub in mutate filter to remove null bytes
- Field mapping conflicts (mapper_parsing_exception) due to different data types for same field names in Elasticsearch indices. Resolved by defining explicit mappings in index templates and applying type conversion filters in Logstash
- Elasticsearch indices switched to read-only mode due to disk watermark exceeded during log volume spikes. Applied ILM (Index Lifecycle Management) policies to auto-delete indices older than 7 days and stabilize disk usage

Results

- 90% log search time reduction (individual Pod access → single Kibana interface)

- 70% reduction in root cause analysis time (integrated log analysis and correlation tracking)
- Real-time processing of 20GB daily log data with 7-day retention

Tech Stack

Elasticsearch, Logstash, Kibana, Filebeat, Kubernetes

Project 4: Developer Productivity Tools (Doc Automation · LLM Service · APK Deploy Bot · Git Mirroring)

Contribution 100% (Full system design & development)   2024.12 ~ 2026.01

Designed, developed, and operated 4 internal tools to automate repetitive manual tasks and boost development team productivity.

4-1. GitLab-Google Drive Document Automation System (2 weeks, 2025.08)

Problem

Dual work of writing technical docs in GitLab markdown then manually uploading to Google Drive

Solution

Integrated GitLab Webhook with Google Drive API for auto-sync on push. Python converts markdown → Google Docs for upload

Results

80% document management time reduction (10min → 2min per doc), unified GitLab as Single Source of Truth

Tech Stack

GitLab Webhook, Google Drive API, Python, Bash Script

4-2. Internal LLM Service Deployment (4 weeks, 2026.01)

Problem

Growing LLM demand from dev team, cost burden and internal data security concerns with external SaaS

Solution

Deployed Ollama and Open WebUI on Kubernetes on on-premises GPU server (RTX 3060). Configured model storage with NFS

Results

Reduced external LLM API costs and eliminated data leakage risks, entire dev team uses for code review, documentation, and debugging

Tech Stack

Ollama, Open WebUI, Kubernetes, NFS, GPU (RTX 3060)

4-3. Slack APK Distribution Automation Bot (4 weeks, 2025.02)

Problem

Manual APK delivery to QA team after build completion caused sharing delays and version confusion

Solution

Developed Python bot that auto-sends build completion messages with download QR codes to Slack when Jenkins APK builds finish, deployed on Kubernetes

Results

90% QA team delivery time reduction, eliminated version confusion with QR code-based instant install, automated build history logging

Tech Stack

Python, Slack Bot API, Jenkins, Kubernetes, Docker

4-4. Git Repository Mirroring Tool Development (4 weeks, 2026.02)

Problem

Manual source code synchronization across AWS CodeCommit, GitLab, GitHub caused omissions and delays

Solution

Developed Go-based bidirectional Git mirroring tool (git-bridge). Supports SQS polling (CodeCommit) + HTTP Webhook (GitLab/GitHub) event sources with incremental sync (git fetch) and loop detection. Deployed on Kubernetes with Slack notification integration

Results

100% automation of multi-platform source sync, eliminated manual mirroring, open-sourced and utilized alongside GitHub Actions (multi-git-mirror)

Tech Stack

Go, AWS SQS, GitLab Webhook, GitHub Webhook, Kubernetes, Docker

---

NerdyStar

2023.03 ~ 2024.07

DevOps Engineer

Worked as a DevOps Engineer at a game and blockchain startup, leading the large-scale cloud migration project from AWS to GCP.

Transitioned to leverage GCP Startup Program cost optimization and GCP's global network performance, successfully completing the full service migration. Operated a dual source management system with GitLab for on-premises and GitHub for production environments.

Project 1: Large-scale AWS → GCP Cloud Migration & CI/CD Reconstruction

Contribution 70-100% (Infra design/migration lead 70%, CI/CD pipeline 100%)   7 months (2023.05 ~ 2023.12)

Problem

AWS costs were continuously rising ($10,000+/month), particularly NAT Gateway and data transfer costs. GCP's global load balancing was needed for multi-region game services, and the existing Jenkins-based deployment system had complex configuration management with no deployment history tracking.

Approach

Established a 3-phase migration strategy executed sequentially. Phase 1: Built and validated dev environment on GCP. Phase 2: Migrated QA environment. Phase 3: Migrated production. Codified GCP infrastructure with Terraform (100% excluding IAM), then rebuilt GitOps-based CI/CD pipeline combining GitLab CI and ArgoCD. Standardized environment configs with Helm Charts and established Git commit-based deployment history management.

Troubleshooting

- Traffic routing issues during AWS ALB to GCP Global LB transition due to health check and backend service configuration differences. Analyzed GCP NEG (Network Endpoint Group) structure and reconfigured for GKE workloads
- Data corruption during AWS RDS to Cloud SQL migration due to character set differences. Resolved by writing pre-validation scripts and implementing step-by-step data integrity check processes
- Network design issues due to AWS vs GCP subnet model differences (AZ-based vs region-based). Redesigned CIDR blocks and converted firewall rules to GCP tag-based approach

Results

- 50% cloud cost reduction ($10,000 → $5,000/month), ~$60,000 annual savings
- Completed service migration with 3-phase strategy
- Established IaC management for entire GCP infrastructure with Terraform
- 50% deployment time reduction (40min → 20min), 95% rollback time decrease (30min → 1-2min)
- 3x weekly deployments (2 → 6), 100% deployment history tracking via Git commits

Tech Stack

GCP (GKE, Cloud SQL, Cloud Storage, VPC, Global LB 등), Terraform, GitLab CI, ArgoCD, GitHub Actions, Kubernetes, Helm, Docker

---

Project 2: Game Data Collection Automation System

Contribution 100% (Python script development & operations)   3 months (2024.01 ~ 2024.03)

Problem

Game and blockchain data was collected manually, with analysts spending 2-3 hours daily on data collection and frequent data gaps due to human errors.

Approach

Developed Python automation scripts on Cloud Functions calling game APIs and blockchain RPCs for data collection. Scheduled periodic execution via Cloud Scheduler and used Google Sheets API to auto-load collected data into spreadsheets for immediate analysis by the analytics team without additional tools.

Results

- 95% data collection automation (eliminated 2-3h daily manual work)
- 10x data throughput increase (10GB → 100GB daily)
- 0% data gap rate through automation eliminating human errors

Tech Stack

Python, Cloud Functions, Cloud Scheduler, Google Sheets API, Docker

Project 3: PLG Stack Monitoring System

Contribution 100% (System design & implementation)   1 month (2024.04)

Problem

No real-time monitoring system for Kubernetes clusters and applications, allowing only reactive incident response with excessive time spent on root cause analysis.

Approach

Built a PLG Stack collecting metrics with Prometheus and logs with Loki, unified visualization through Grafana dashboards. Real-time monitoring of CPU/memory/network usage and application logs, with automatic Slack alerts on threshold breaches.

Results

- 80% proactive incident detection rate through threshold-based alerts
- 60% incident response time reduction through real-time monitoring and alerting
- Service availability improved from 99.5% to 99.9%

Tech Stack

Prometheus, Loki, Grafana, Promtail, Fluent-bit, Slack API

Iorchard

2022.04 ~ 2023.03

Infra Engineer

Responsible for building PaaS (Kubernetes + OpenStack + Ceph) infrastructure solutions for clients and providing technical support. Designed and built on-premises cloud infrastructure for financial and public sector clients, and conducted training for client operations teams.

Project 1: Customized PaaS Solution for Clients

Contribution 100% (Infra design & Kubernetes cluster build)   11 months (2022.04 ~ 2023.03)

Problem

Each client had different environment and server spec requirements, making standardized solutions insufficient.

Approach

Designed Kubernetes clusters in HA (High Availability) configuration per client requirements and built virtual machine management environments with OpenStack. Provided unified block/file/object storage through Ceph and conducted parallel infrastructure operations training for client teams.

Results

- Successfully built and delivered PaaS solutions to 5 clients
- Achieved 99.9% availability with Kubernetes cluster HA configuration
- Reduced per-client deployment time through Ansible-based build automation

Tech Stack

Kubernetes, OpenStack, Ceph, Ansible, Rocky Linux/CentOS

Project 2: Kubernetes Operations Training Program for DP

Contribution 100% (Training design & delivery)   6 months (2022.06 ~ 2022.11)

Problem

DP's operations team lacked Kubernetes and container-based infrastructure experience, anticipating difficulties in self-operation after PaaS solution adoption.

Approach

Designed a step-by-step training curriculum from Kubernetes basics to cluster operations and troubleshooting, with hands-on lab environments for practice-oriented training. Also conducted parallel training on OpenStack and Ceph storage operations to ensure full PaaS stack operational capability.

Results

- Established DP's autonomous Kubernetes operations system
- Reduced client technical inquiries post-training, achieved self-incident response capability
- Standardized training materials reused for subsequent client trainings

Tech Stack

Kubernetes, OpenStack, Ceph, Ansible, Rocky Linux/CentOS

Project 3: Kubernetes Certificate Auto-renewal Process Improvement

Contribution 100% (Solo)   2 weeks (2022.11)

Problem

Kubernetes cluster certificates expired annually, requiring ~5 hours of renewal work per client per year, with service outage risks upon expiration.

Approach

Analyzed the Kubernetes certificate management process and modified kubeadm settings to extend certificate validity from 1 year to 10 years. Developed automation scripts applicable to existing clusters and deployed to all clients.

Results

- 10x certificate renewal cycle extension (annual → once per 10 years)
- ~50 hours annual operations savings across 10 clients
- Eliminated outage risks from certificate expiration

Tech Stack

Kubernetes, kubeadm, Bash Script, OpenSSL